
ECOSISTEMAS DE SISTEMAS DISTRIBUIDOS
Tipos de sistemas distribuidos
Arquitectura Cliente - Servidor: El modelo tradicional donde las responsabilidades se dividen entre servicios y solicitantes [4].
Computación en la red: Paradigma que permite compartir recursos geográficamente dispersos entre múltiples dominios [6].
Sistemas de Par a Par: Arquitecturas descentralizadas donde cada nodo puede actuar como cliente y servidor a la vez, mejorando la escalabilidad [4].
Computación en la niebla y el borde: Despliegues que acercan el procesamiento y el almacenamiento a la fuente de datos para reducir la latencia [7].
Arquitecturas de N-Capas: Evolución del modelo cliente-servidor que separa la presentación, la lógica de negocio y el almacenamiento de datos en capas distintas [4].
Sistemas basados en eventos (EDA): Arquitectura donde el flujo de información es impulsado por la producción y detección de eventos [8].
Microservicios: Un enfoque donde la aplicación se divide en servicios pequeños y autónomos que se comunican mediante protocolos ligeros [5].
Modelos de Comunicación
Mecanismos con los que los procesos de un sistema distribuido intercambian información y coordinan sus acciones [9].
Comunicación síncrona
El emisor envía una solicitud y se bloquea esperando la respuesta del receptor antes de continuar [9].
RPC (Llamada a procedimiento remoto)
Invoca un servicio en otra máquina con una semántica similar a una llamada local [9].
gRPC y JSON-RPC
Protocolos modernos de RPC que estructuran solicitudes y respuestas; JSON-RPC es el usado por MCP [10].
Comunicación asíncrona
El emisor continúa su ejecución sin esperar respuesta; emisor y receptor no necesitan coincidir en el tiempo [11].
Message-Oriented Middleware (MOM)
Middleware que comunica procesos mediante colas de mensajes intermedias, sin que emisor y receptor estén activos simultáneamente [11].
Publicación/Suscripción (Pub/Sub)
Los publicadores envían mensajes por tema sin conocer a los suscriptores, que reciben solo los temas de su interés [12].
Protocolos Gossip
Difusión epidémica de información entre nodos P2P, con alta tolerancia a fallos y sin coordinador central [13].
Tolerancia a Fallos y Confiabilidad
Mantiene el funcionamiento del sistema ante fallos parciales y permite recuperar sus servicios.
Redundancia y replicacion
Mantiene copias de datos y servicios en distintos nodos para garantizar continuidad ante fallos [8].
Checkpointing
Guarda periódicamente el estado del sistema para recuperarlo después de un fallo [19].
Circuit Breakers
Detienen temporalmente solicitudes a servicios defectuosos para evitar fallos en cascada [10].
Timeouts
Detectan posibles fallos cuando un nodo no responde dentro del tiempo establecido [20].
Tolerancia a Fallos Bizantinos
Permite alcanzar acuerdos incluso cuando algunos nodos actúan maliciosamente o envían información contradictoria [21].
Teorema CAP y Modelos de Consistencia
Establece límites para garantizar consistencia y disponibilidad durante particiones de red.
Consistencia (C)
Cada lectura devuelve los datos más recientes o un error [22].
Disponibilidad (A)
Cada solicitud a un nodo operativo recibe respuesta, aunque los datos no sean recientes [23].
Tolerancia a Particiones (P)
Permite operar pese a interrupciones de comunicación entre grupos de nodos [24].
Middleware
Interoperabilidad
Permite la comunicación entre sistemas y plataformas diferentes [16].
Gestión de recursos
Facilita el descubrimiento de servicios y la distribución de cargas [17].
Seguridad
Incorpora mecanismos de autenticación y protección de datos [18].
Abstracción
Oculta la complejidad técnica de la comunicación entre componentes distribuidos [19].
Transparencia
Acceso
Permite utilizar recursos locales y remotos de la misma manera [4].
Ubicación
Oculta la localización física de los recursos [4].
Replicación
Oculta la existencia de múltiples copias de datos o servicios [4].
Algorítmica y de IA
Busca que las decisiones automatizadas sean comprensibles y auditables [3].
Fallos
Busca mantener el funcionamiento del sistema ante fallos de sus componentes [20].